

DeepFuzz: Documentation-Driven Fuzz Testing for Deep-Learning APIs

From API documentation to executable seeds, generated tests, coverage evidence, and bug triage for JAX, TensorFlow, and PyTorch

Aryan Gupta

UQ School of Electrical Engineering and Computer Science

Poster claim

DeepFuzz turns natural-language API documentation into reproducible fuzzing artefacts: specs, seeds, pytest files, coverage reports, and strict triage evidence.

1,992

accepted documentation APIs

1,407

frozen generated pytest tests

12,383

valid generated programs

190

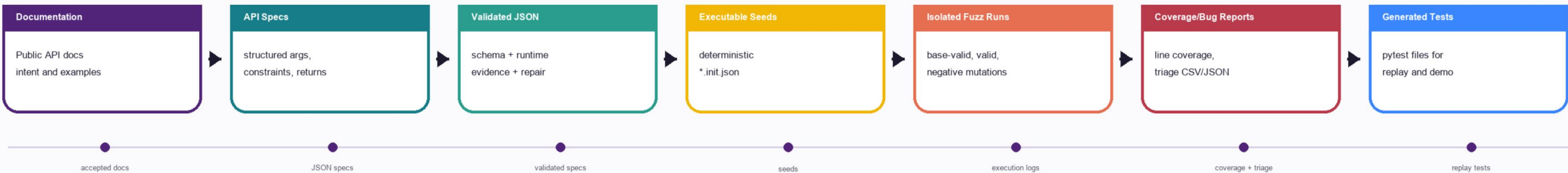
candidate implementation-bug records

1,209

device-oracle mismatch records

End-to-end DeepFuzz evidence pipeline

Each stage leaves an auditable artefact that can be inspected, replayed, or turned into generated tests.



Motivation / Problem

- Deep-learning APIs are large, fast moving, and hard to test manually.
- Documentation captures intended behaviour but is not directly executable.
- Manual examples rarely exercise edge values, invalid inputs, or device differences.

Research Aim

Convert public API documentation into deterministic fuzz tests and auditable evidence across JAX, TensorFlow, and PyTorch.

Why Existing Testing Is Hard

- Library docs mix prose, examples, constraints, and partial error behaviour.
- Executability depends on installed versions, tensor types, shapes, and runtime devices.
- Whole-library coverage under binary wheels hides the behaviour of selected APIs.

Innovation And Contribution

- Documentation-first, runtime-grounded test generation.
- Multi-stage validation and local LLM repair before fuzzing.
- Reproducible run manifests and per-API artefact trails.
- Selected-function coverage avoids misleading whole-library claims.
- Strict triage separates bugs, doc mismatches, expected rejections, and pipeline failures.

Results From Final Thesis Runs

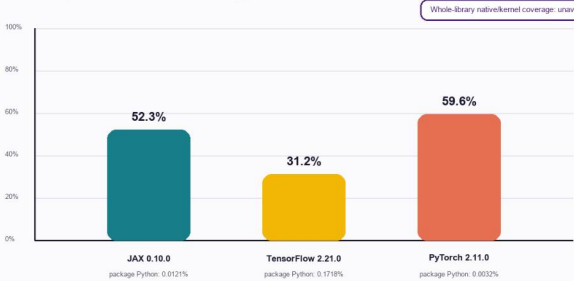
Source of truth: pipeline_runs/[jax,tensorflow,torch]/*-thesis-all and stage4/results/*-thesis-all-coverage.

Library	Accepted	Tests	Base-valid	API exec.	Func cov.	Valid progs	Cand. bugs	Device mm.
JAX 0.10.0	477	379	476/476	100.0%	52.3032%	3,421	2	0
TensorFlow 2.21.0	922	705	697/705	98.8652%	31.2415%	6,866	3	0
PyTorch 2.11.0	593	323	323/323	100.0%	59.5745%	2,096	185	1,209

Native/kernel coverage from binary wheels is recorded as unavailable_on_this_run; the headline metric is selected-function Python line coverage.

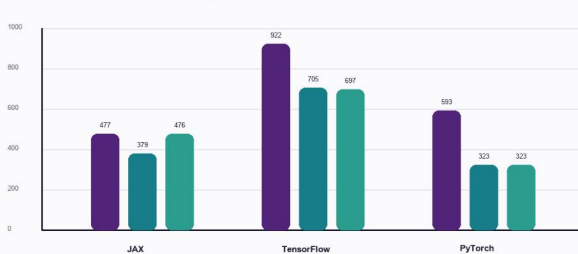
Selected-function Python line coverage

Headline metric: reports lines in the selected API functions reached by generated executions.



API readiness and execution evidence

Accepted documentation candidates are narrowed to frozen pytest files and base-valid executed APIs.



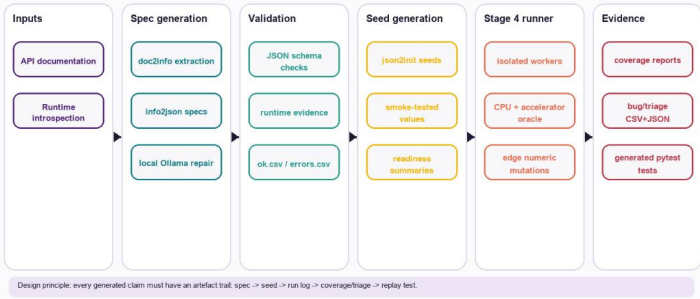
Analysis / Conclusion

DeepFuzz is a complete, reproducible software-engineering pipeline. It reaches every base-valid API in JAX and PyTorch and 98.8652% in TensorFlow; reports combine selected-function coverage with execution, seed, generated-test, and triage evidence.

Implementation Architecture

Implementation architecture

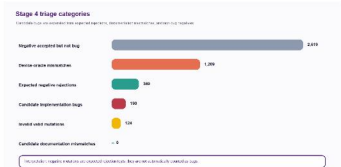
Documentation-first generation is grounded by schemas, smoke tests, isolated runs, and reproducible reports.



Demonstration Artefacts

- Replay one generated pytest for jax.numpy.sin.
- Inspect per-API seed JSON and generated test file.
- Show Stage 4 coverage report and torch bug_report.csv.
- Explain one path: documentation -> spec -> seed -> fuzz run -> evidence.

Triage And Oracles



Device oracle and edge numeric mutations were enabled; candidate records require manual confirmation before upstream reporting.

References: project README, final_report.json, coverage_report.json, bug_report.csv, and run_manifest.json from committed *-thesis-all runs.

CRICOS 00025B • TEQSA PRV12080

Acknowledgements

UQ EECS thesis guidance and assessment rubric; public JAX, TensorFlow, and PyTorch documentation; open-source Python testing and coverage tooling.

uq.edu.au